SO 2022-2023 sem 1: Întrebări examen oral

Aceste întrebări vor fi folosite pentru discuția de la examenul oral de SO. Nu sunt singurele întrebări care vor fi prezente în discuție; de la aceste întrebări discuția va atinge și alte noțiuni prezentate la cursul de SO.

Stiva software

1. **Ce este un apel de sistem?**

   Un apel de sistem este modalitatea prin care un program poate interacționa cu sistemul de operare. În general, apelurile de sistem sunt referite printr-un nume și asociate cu un număr.
   Modalitatea de comunicare între user space şi kernel space.

2. **De ce sunt necesare apeluri de sistem?**

   Apelurile de sistem sunt necesare pentru a interacţiona cu sistemul de operare. Invocarea unui apel de sistem înseamnă schimbarea nivelului de privilegiu de la neprivilegiat la privilegiat.

3. **Ce avantaj / dezavantaje au apelurile de sistem?**

   Avantaj: interacţiunea cu sistemul de operare
   Dezavantaje: pot fi blocante dacă nu sunt folosite corespunzător (ex: read())

4. **Ce înseamnă user/application mode/space (mod neprivilegiat)? Ce înseamnă kernel/supervisor mode/space (mod privilegiat)?**

   User mode(modul neprivilegiat) este modul în care un proces sau o aplicație este rulată. În acest mod, procesul are acces limitat la resursele sistemului de operare şi nu poate efectua operațiuni care pot afecta funcționarea acestuia. Fiecare proces are câte un virtual address space (VAS) separat.
   Bitul este setat pe 1 în user mode.

   Kernel mode(modul privilegiat) este modul în care sistemul de operare rulează. În acest mod, sistemul de operare are acces la toate resursele hardware si software. Acest lucru este necesar pentru a permite sistemului de operare să gestioneze resursele. În acest mod, de obicei, ruleaza doar sistemul de operare şi driverele.
   Bitul este setat pe 0 în kernel mode.
   Sistemul porneşte în **kernel mode** până când este încărcat sistemul de operare, moment în care se trece în **user mode**.

5. **Cum se realizează tranziția din mod neprivilegiat în mod privilegiat?**

Tranziția în mod privilegiat se realizează în urma unui apel de sistem, modificându-se bit-ul de mod din 1 în 0. După executarea apelului de sistem, se trece înapoi în user mode (mod neprivilegiat).

6. **Ce se întâmplă în momentul tranziției în mod privilegiat? Cum se/Cine asigură (enforcement) existența modului privilegiat?**

   Bit-ul de mod se schimbă din 1 în 0 când se trece în modul privilegiat(sau invers). Existența acestui mod este asigurată de mecanismele de securitate implementate în sistemul de operare. (ex: MMU = Memory Management Unit).

7. **Ce este o bibliotecă?**

   O bibliotecă este o colecție de implementări, scrise în termenii unui limbaj de programare, care are o interfață bine definită. De exemplu se poate folosi o bibliotecă pentru a efectua apeluri de sistem, în loc sa fie scrisă de fiecare dată când este nevoie de unul. (formulare proasta)

8. **Care este diferența dintre o aplicație și o bibliotecă?**

   O aplicație este un program care rulează pe un sistem de operare și este concepută pentru a efectua o anumită sarcină sau a oferi o anumită funcționalitate utilizatorului. O bibliotecă este o colecție de cod sursă sau cod compilat care poate fi folosită de o aplicație pentru a efectua diferite operații sau pentru a accesa anumite resurse.

9. **Care este asocierea apel de bibliotecă / apel de sistem?**

   Apelurile de bibliotecă pot fi construite pentru sau peste apeluri de sistem:
   -de exemplu apelul de bibliotecă open este construit pentru apelul de sistem open
   -apelul de bibliotecă fopen este construit peste apelul de sistem open
   Există și apeluri de bibliotecă care nu au legătură cu apelurile de sistem.(ex: strcpy)

10. **Ce este entry point-ul într-un executabil?**

    Entry point-ul unui executabil este zona de memorie unde începe execuția codului programului atunci când acesta este rulat. Este prima instrucțiune pe care o va executa procesorul la rularea programului.

11. **Care este rolul bibliotecii standard C (libc)?**

    Apelurile de sistem sunt “wrapped” de biblioteca standard C. Această bibliotecă implementează atât funcții standard C cât și funcții **POSIX**(**Portable Operating System Interface**). În mod normal, un program nu poate invoca apeluri de sistem deoarece compilatorul nu le recunoaște. Biblioteca libc leagă aceste apeluri de sistem cu apeluri de bibliotecă.

12. **Ce acțiuni se pot executa doar în mod privilegiat?**

Apelurile de sistem se pot executa doar în mod privilegiat, altfel aceste apeluri ar fi doar niște apeluri de bibliotecă. **Comunicarea cu I/O**, **mapările și alocările de memorie**, managementul timerelor și al întreruperilor, **comunicarea cu alte procese**, **configurarea de permisiuni**, pornirea și oprirea aplicațiilor.

13. **Ce operații / instrucțiuni low-level (ISA) se pot executa doar în mod privilegiat?**

ISA = instruction set architectures (part of the abstract model of a computer that defines how the CPU is controlled by the software)

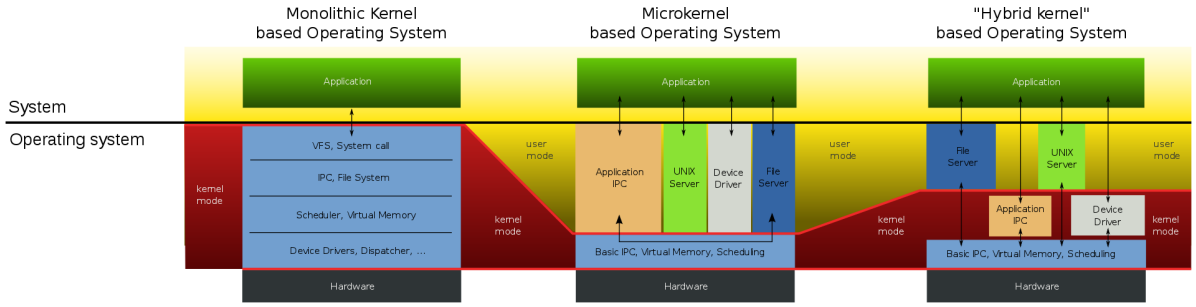
Operațiile low-level care se pot executa doar în mod privilegiat sunt **load** și **store**(mov).

14. **Ce este un sistem de operare monolitic?**

Un sistem de operare monolitic are și **kernel space și user space**, doar că se întâmplă mai multe chestii în kernel space. Toate driverele sunt incorporate in kernel, ele rulând în supervisor mode. Un driver care nu merge bine poate să corupă BIOS-ul care pornește înaintea sistemului de operare. Un driver corupt => system is down.
La început, UNIX avea structura monolitică.
(Linux ???, Solaris, Microsoft Windows 95/98/Me, MacOS până la MacOS 8.6 etc.)

Un sistem de operare monolitic este un sistem în care user services și kernel services sunt implementate în același spațiu de adrese (motiv pentru care este eficient). Este mai puțin flexibil, pentru ca atunci cand se face o modificarea asupra unei componente, trebuie schimbat tot. Este un sistem mai mare, intrucat kernelul este mai mare. Dacă un serviciu pica, pica toate. Toate functiile SO-ului se realizeaza prin apeluri de sistem.

In monolitic totul se intampla in kernel mode.
*A **monolithic kernel** is an operating system architecture where the entire operating system is working in kernel space. (de pe wikipedia).*



15. **Ce este un sistem de operare de tip microkernel?**

Un sistem de operare de tip microkernel are atât kernel space cât și user space. Microkernelul a eliminat componentele esențiale din **kernel-space** și le-a implementat în **user-space** sub formă de programe.
(HelenOS, Zircon, Horizon, Redox etc.)

Un sistem de operare de tip microkernel are user services si kernel services implementate în locuri diferite din memorie (motiv pt care este mai lent, serviciile fiind nevoite sa comunice între ele). Se poate extinde ușor, kernelul este mai mic. Dacă un serviciu, fie el kernel sau user, pica, nu afectează alte servicii.

16. **Care sunt avantajele unui sistem de operare monolitic?**

    Avantajele unui sistem monolitic sunt: execuție mai rapidă, necesită mai puțin cod. (overhead spatial si temporal)

17. **Care sunt avantajele unui sistem de operare de tip microkernel?**

    Avantajele unui sistem de tip microkernel sunt: securitatea, acesta este modular și se extinde mai ușor.

    Windows, linux și macos sunt hybrid.

18. **Care tip de sistem de operare are mai multe apeluri de sistem?**

    Microkernel-ul are mai multe apeluri de sistem (cred) deoarece sunt mai multe tranziții din user space în kernel space pentru a permite programelor să interacționeze cu hardware-ul.

19. **Care este avantajul folosirii mașinilor virtuale din perspectiva securității?**

    Mașina virtuală este un mediu relativ izolat de sistemul de operare. O mașină virtuala permite rularea unui sistem de operare în alt sistem de operare. Mașinile virtuale pot rula orice sistem de operare pe când un **container** poate rula doar sistemul host.

20. **Ce este o bibliotecă statică? Ce este o bibliotecă dinamică?**

    Biblioteca statică este încărcată în executabil la compilare. Codul bibliotecii se va regăsi în executabilul final, acesta fiind mai mare.
    Biblioteca dinamică este încărcată la load-time(doar adresele) în executabil, apoi la rularea executabilului este încărcată în memorie. Executabil mai mic.

21. **Când preferăm folosirea static linking, respectiv dynamic linking? Cu ce diferă un executabil dinamic de un executabil static?**

    Prin **static linking** se obține un executabil static, de dimensiune mare (sunt încărcate toate bibliotecile în executabil la compilare). Procesul obținut dintr-un executabil static nu poate partaja memorie cu alte procese (fără ajutor extern).

Prin **dynamic linking** se obține un executabil dinamic, de dimensiuni reduse(este încărcată doar adresa bibliotecii în executabil). Procesele obținute dintr-un executabil dinamic partajează toate zonele read-only(text și rodata).

Un **executabil static** este mai rapid deoarece nu face linking la load-time și este mai portabil deoarece are toate funcțiile necesare rulării incluse în executabil (previne dependency hell - "software packages which have dependencies on specific versions of other software packages").
Un **executabil dinamic** ocupă mai puțină memorie deoarece nu are toată biblioteca încărcată în executabil. De asemenea, mai multe programe pot pointa spre aceeași bibliotecă dinamică din memoria RAM.

22. Dați exemplu de apel de sistem blocant.

Exemple de apeluri de sistem blocante:
-**read()** se blochează dacă bufferul este gol;
-**write()** se blochează dacă bufferul este plin;
-**recv()** este un apel blocant deoarece acesta citește un mesaj de pe socket, datele pot fi primite și după o oră.

23. Dați exemplu de apel de sistem neblocant.

Exemple de apeluri de sistem neblocante:
-**open()** deschide un fisier(în cazul în care fișierul nu există și este specificat flagul O_CREAT se și creează fișierul). Acesta returnează file descriptor-ul fișierului.
-**close()** închide file descriptor-ul și șterge referința programului către acesta. În cazul în care file descriptorul este ultimul care pointează la file descriptorul deschis de open, atunci toate resursele asociate cu file descriptorul curent se eliberează (sau ceva de genu). Dacă s-a folosit unlink pe ultimul file descriptor, fișierul se va șterge în urma apelării close.
-**getpid()** returnează process id-ul
-**seek()** poziționează cursorul read/write la offsetul specificat.

24. De ce, în general, o aplicație trebuie să execute un apel de sistem pentru a accesa un dispozitiv hardware? De ce NU poate accesa direct dispozitivul hardware?

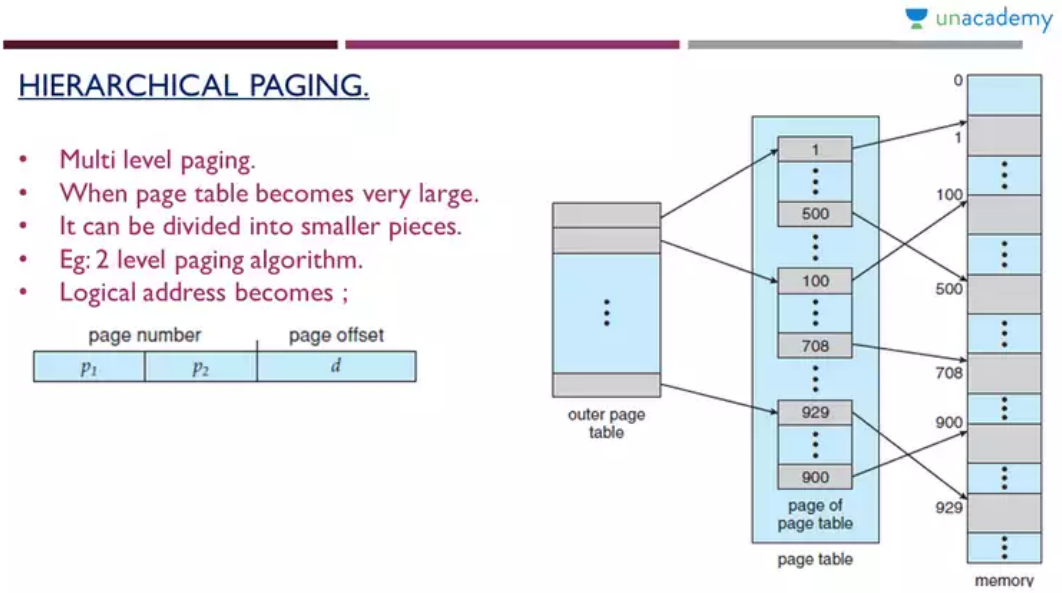
O aplicație trebuie să execute un apel de sistem pentru a accesa un dispozitiv hardware deoarece aplicația rulează în user-space unde nu are acces la hardware. Toate aplicațiile se află în user-space din motive de securitate.

25. Ce înțelegem prin overhead spațial și overhead temporal?

Overhead spațial: câte resurse de spațiu ai nevoie pentru a executa o acțiune
Overhead temporal: câte resurse de timp sunt necesare pentru a executa o acțiune

26. Dați exemplu de mecanism / funcție care reduce overhead-ul spațial și unul care reduce overhead-ul temporal.

Tabela de pagini ierarhică reduce **overhead-ul spațial**.



TLB (Translation look-aside buffer) reduce **overhead-ul temporal**. Când un proces intră pe procesor (mai exact, thread-urile lui), adresele procesului sunt încărcate în **TLB**.

27. **Ce înseamnă double buffering? În ce situație concretă (mecanism/funcție) apare?**

    Double buffering este tehnica prin care două buffere sunt folosite pentru creșterea performanței. În timp ce se lucrează datele dintr-un buffer se pot pregăti datele în altul.
    Double buffering se folosește în situația producer-consumer (producătorul scrie iar consumer-ul citește).

28. **Ce se întâmplă când există o eroare critică (de tip Segmentation fault) la nivelul sistemului de operare?**

    În cazul în care se încearcă accesarea unei zone de memorie invalidă, se întoarce eroarea critică **segmentation fault** și se închide programul. La nivelul kernel-ului se încearcă rezolvarea erorii prin apelarea unei rutine din sistemul de operare. În cazul windows-ului apare Blue Screen.

29. **Dați exemplu de un apel de bibliotecă standard C care nu cauzează apel de sistem.**

    Apeluri de bibliotecă care nu cauzează apeluri de sistem:
    -**strpy()**
    -**strlen()**

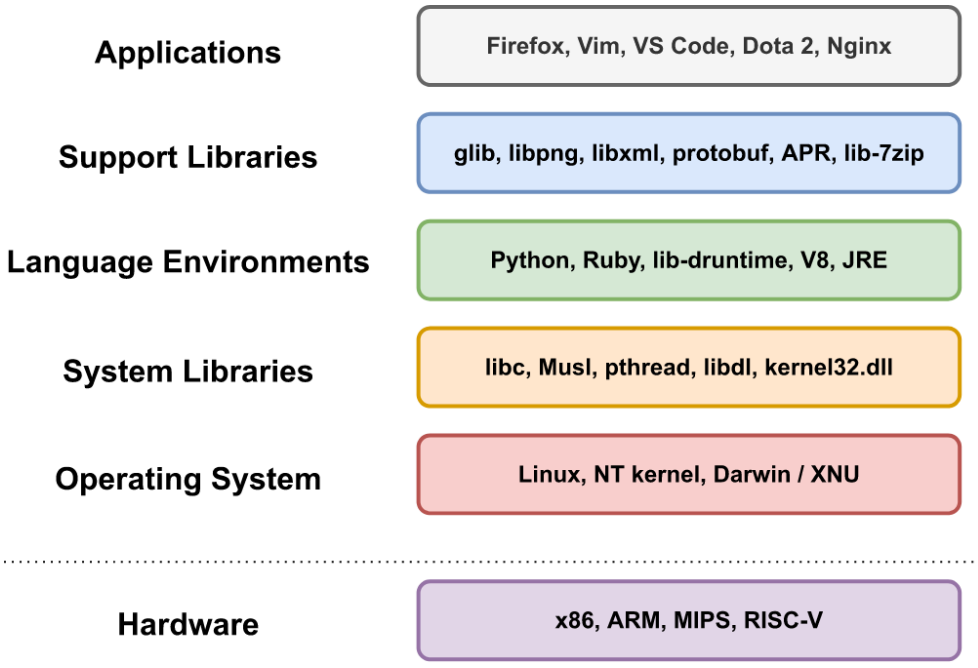
30. Dați exemplu de un apel de bibliotecă standard C care cauzează apel de sistem.

Apeluri de bibliotecă care sunt implementate peste un apel de sistem:
-**write()**
-**fopen()**
-**open()**
-**close()**
-**scanf()** & **printf()** - cheamă read & write (nu cauzează mereu un apel de sistem)

31. Care este avantajul și dezavantajul existenței unei stive software la nivelul unui sistem de calcul?

Avantajul: Stiva software poate fi folosită pentru a ajuta la gestionarea resurselor sistemului și a structurii de date. De asemenea, poate servi ca un mecanism de control pentru a asigura o execuție corectă a programului și a limita recursivitatea.

Dezavantaj: Stiva software poate fi vulnerabilă la atacuri din afară. De asemenea, poate fi foarte consumatoare de memorie, mai ales în cazul în care este folosită în mod intensiv. Uneori, poate fi dificil de implementat și poate să nu ofere cele mai eficiente soluții de programare.



32. Care este avantajul și dezavantajul folosirii unui limbaj de programare high-level (precum Java, Python, Lua)?

Avantaje:

-ușor de folosit
-ușor de înțeles
-ușor de făcut debug
-gestionarea memoriei nu mai este o problemă a programatorului

Dezavantaje:
-incet
-puțin acces la hardware
-ocupă mai multă memorie

33. **Care este avantajul și dezavantajul folosirii unui limbaj de programare low-level (precum C, C++, Rust, D)?**

    Avantaje:
    -acces facil la hardware
    -rapid și eficient dpdv al memoriei

    Dezavantaje:
    -nu sunt portabile
    -greu de scris cod și de făcut debug
    -predispus la memory leak-uri

Date

1. **Cum asigură sistemul de operare separația între procese?**

   Prin mecanismul de memorie virtuală. Fiecare proces are impresia controlului întregii memorii. Îi este imposibil să acceseze memoria altui proces.

2. **Ce înseamnă mecanismul de memorie virtuală?**

   Mecanismul de memorie virtuală este un mod al sistemului de operare de a împărți memoria fizică între procese și de a asigura izolarea acestora. Acesta are grijă ca procesele să nu utilizeze mai mult spațiu decât spațiul fizic. Prin utilizarea memoriei virtuale, fiecare proces se crede singur și crede că are toată memoria disponibilă.

3. **Ce reprezintă spațiul virtual de adrese al unui proces?**

   VAS(Virtual address space) este unic fiecărui proces. Acesta reprezintă zona de memorie virtuală a procesului.
   VAS = set de intervale de adrese virtuale pe care un sistem de operare le pune la dispoziție unui proces.

4. **Cu ce diferă zona de date de zona de cod în spațiul virtual de adrese al unui proces?**

Zona de cod(.text) este read execute aceasta rămânând neschimbată pe tot parcursul rulării.
Zona de date (conține variabile statice inițializate) este read-write și se poate schimba în timpul rulării.

5. **Ce permisiuni are zona .text/.data/.rodata/.bss/de stivă/de heap?**

-**text**(cod): read-execute;
-**rodata**(constante): read-only;
-**data**(variabile statice inițializate): read-write;
-**BSS**(variabile statice neinițializate, și inițializate cu 0): read-write;
-**heap**(zona pentru alocări dinamice): read-write;
-**stack**(variabile locale): read-write.

6. **Ce este paginarea memoriei?**

Compartimentarea spațiului virtual al fiecărui proces și al spațiului fizic al sistemului în componente de dimensiune fixă(de obicei 4k) numite pagini:
-pagini virtuale care se numesc **pages**
-pagini fizice care se numesc **frames**
Paginarea memoriei e cea care generează fragmentarea internă.

7. **Ce rol are tabela de pagini?**

Tabela de pagini conține adresa paginii fizice și e indexată cu adresa paginii virtuale.

8. **Ce este și ce rol are MMU (Memory Management Unit)?**

MMU ia adresele logice și le transformă în adrese fizice. Verifică dacă adresa de memorie este validă și dă eroare în caz contrar, blochează procese și ocupă ⅓ din procesor.

9. **Ce rol are TLB (Translation look-aside buffer)?**

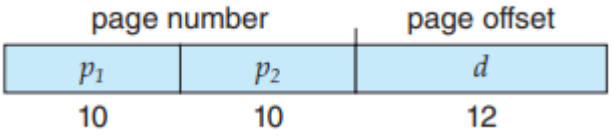
TLB conține cele mai recente accesări la intrările în tabela de pagini. TLB este o memorie de tip cache. Fiecare intrare din TLB conține două intrări: o cheie și o valoare.

(TLB (Translation look-aside buffer) reduce **overhead-ul temporal**. Când un proces intră pe procesor (mai exact, thread-urile lui), adresele procesului sunt încărcate în **TLB**.)

10. **Care este ordinul de mărime al numărului de intrări ale TLB?**

În mod normal, TLB are undeva între 32 și 1024 de intrări.

11. **Ce conține o intrare în tabela de pagini?**



Conține adresa fiecărei pagini din memoria fizică și offsetul locației frame-ului căutat.

12. Ce înseamnă tabelă de pagini multi-nivel (ierarhică)? De ce este utilă?

Tabela de pagini ierarhică reprezintă întreaga tabelă de pagini dar în funcție de arhitectură și se folosește pentru a reduce overhead-ul spațial.
Pentru sistemele de operare pe 32 bits conține două niveluri iar pe 64 bits conține 4 sau 5 niveluri.
Tabela de pagini ierarhică are undeva la 2^10 intrări.

13. Când are loc un TLB miss?

Un TLB miss are loc atunci când:
-Nu este găsită pagina, adresa.
-Nu e mapată adresa virtuală.
-Adresa virtuală e mapată la adresa fizică dar nu corespunde în TLB.
-Tabela de intrare a TLB-ului este plină și nu există intrări libere pentru a o mapa.

De obicei un TLB miss are un impact negativ deoarece trebuie accesată tabela de pagini care este mai lentă decât TLB-ul.

14. De ce se golește TLB-ul (TLB flush) la schimbare de context?

TLB flush se dă pentru a asigura că următorul proces nu folosește adresele procesului trecut.

15. De ce nu este nevoie de TLB flush la schimbarea de context între două thread-uri ale aceluiași proces?

Nu este nevoie deoarece threadurile aparțin aceluiași proces, deci au același spațiu de adrese virtuale (VAS). La thread-uri diferă doar stiva și TLS-ul (Thread local space - the method by which each thread in a given multithreaded process can allocate locations in which to store thread-specific data.)(e un fel de variabila globală a threadului respectiv)

16. Ce înseamnă mecanismul de copy-on-write?

Nu se alocă memorie fizică până când procesul scrie. Vezi și următoarea întrebare.

17. Dați exemple de situații în care are loc mecanismul de copy-on-write.

Când se cheamă apelul de sistem **fork()** se crează un proces copil nou care are spațiul virtual diferit de părinte iar în momentul în care unul dintre ei scrie, se crează un spațiu fizic nou pentru copil.
Pentru apelul **vfork()** se face același proces de mai sus doar că pentru pagini virtuale.

18. Cu ce apel de sistem asociem copy-on-write?

fork()

19. Când se duplică o pagină marcată copy-on-write?

Când unul dintre procese scrie.

20. Cine detectează un acces de scriere într-o pagină marcată copy-on-write?

Sistemul de operare folosește tehnologia de monitorizare a adreselor de memorie pentru a detecta un acces de scriere la o adresă de memorie.

21. Ce înseamnă demand paging?

Este o metodă de management a memoriei virtuale în care se încarcă pagina doar atunci când este accesată. Dacă nu există se dă page fault și se creează pagina.

22. În ce situație apare page fault fără a cauza segmentation fault?

Page fault apare fără a cauza segmentation fault în următoarele cazuri:
- demand paging
- copy-on-write
- în cazul unei pagini virtuale valide al cărui conținut este în **swap.**

23. Ce rol are spațiul de swap?

Swap-ul este o zona de pe disc folosită pentru a aloca mai multă memorie fizică decât este disponibilă. Unele pagini se pun în swap dacă nu există suficientă memorie fizică pentru a face altor pagini loc.

24. Când are loc swap in și swap out?

Swap in are loc când pagina de pe disk este necesară în execuția procesului iar o pagina din memoria fizică este scoasă prin **Swap out** dacă va mai fi necesară.
Deci pt proști = swap in de pe disk in memorie, swap out din memorie pe disk

25. Care este rolul unui page fault? În ce condiții apare?

Page fault este o eroare care apare pentru a preveni accesul într-o zonă de memorie invalida.
Page fault **minor** când este pagina în memorie dar nu este mapată.

Page fault **major** atunci când pagina se află în swap.
Cazuri:

- pagina e nemapata(swapped out/on demand paging)
- pagina nu e alocată => seg fault
- permisiuni (copy-on-write și read only)

26. Care sunt secțiunile/zonele din spațiul de adrese al unui proces?

-**text**(cod);
-**rodata**(constante);
-**data**(variabile statice inițializate);
-**bss**(variabile statice neinițializate, și inițializate cu 0);
-**heap**(zona pentru alocări dinamice);
-**stack**(variabile locale);
-**biblioteci**(cu subzone: text, rodata, data, bss).

27. Ce secțiuni ale unui executabil se pot inspecta doar în timpul rulării?

Heap, stack.
Analiza dinamică: acțiunile analizei statice se pot realiza și în analiza dinamică. La analiza dinamică se poate urmări fluxul execuției unui program: memoria, regiștrii programului, resursele folosite.

28. Care sunt zonele writable din spațiul de adrese al unui proces?

Stack, heap, bss, data

29. De ce sunt avantajoase bibliotecile dinamice pentru spațiul de adrese al unui proces?

Deoarece vor partaja zone de cod din biblioteca dinamică cu alte procese. Bibliotecile dinamice sunt încărcate o singură dată în RAM și mapate în VAS-ul fiecărui proces.

30. Două procese sunt pornite din același executabil, ce zone din spațiul de adrese vor partaja?

Text, rodata, bibliotecile dinamice.

31. Se alocă un buffer a[100]. De ce a[105] NU va rezulta, în general, în Segmentation fault?

În general, în 90% din cazuri, în C nu va genera segfault deoarece pagina a fost alocată/rezervată. Se dă segfault în momentul în care se accesează o pagină nerezervată (nemapată / nu are memorie alocată). O pagină are 4096 bytes.

32. În ce situație a[300] rezultă în Segmentation fault?

Atunci când se accesează o pagină nerezervată (nemapată / nu are memorie alocată).

33. **Câte pagini ocupa char a[100] pentru sistem standard cu pagini de 4096 de octeți?**

    O singură pagină dacă s-a scris în ea sau 0 altfel.

34. **Câte pagini fizice alocă un apel mmap() care alocă 1MB? O pagină ocupă 4KB.**

    În general, o pagină fizică se alocă doar în momentul scrierii. Apelul mmap va ocupa 0 pagini fizice.

35. **Câte pagini fizice alocă un apel calloc() care alocă 1MB? O pagină ocupă 4KB.**

    Depinde. Se vor număra doar paginile care nu sunt noi.

36. ***Ce înseamnă maparea unui fișier în memorie?* De ce este avantajos să mapăm fișiere față de folosirea read/write?**

    Se alocă spațiu pentru fișier.

    Este avantajos deoarece nu se mai fac apeluri de sistem (overhead temporal scăzut) și nu se mai alocă buffere în user-space pentru apelurile read și write (overhead spațial scăzut). În schimb, dezavantajul este că trebuie știută dimensiunea exactă a fișierului.

37. **Câte page fault-uri se pot obține în cazul operației *a = b?**

Calin e transpirat rau frfr on god no cap

    Se pot obține următoarele page faulturi:
    - variabila **a** nu a mai fost accesată (page demanding)
    - dereferențierea lui **a** poate cauza un page fault
    - instrucțiunea în sine poate fi într-o zonă nealocată (RD moment)
    (Posibil mai multe)

38. **Care este numărul maxim de page fault-uri pe care îl poate genera expresia a = b + c?**

    Se poate obține un singur page fault pentru instrucțiuni în sine.

39. **Ce informații sunt reținute în stivă? Ce variabile C?**

    În stivă se rețin **variabilele locale funcției** și **adresele de retur ale funcției**.

În stiva se reține date, variabile și adrese de memorie. Variabilele C pot fi reținute în stivă, cum ar fi variabilele primare (int, char, float, etc.), variabilele globale și variabilele de instanțiere. -> chatgpt

40. Ce se întâmplă la faza de loading (încărcarea unui executabil în memorie și crearea unui proces)?

- alocarea unei zone de memorie pentru încărcarea executabilului.
- copierea executabilului în zona de memorie alocată.
- încărcarea bibliotecilor dependente.
- începerea execuției programului.

41. Ce înseamnă deturnarea fluxului de execuție a unui program (control flow hijack)? De ce este acest lucru relevant pentru un atacator?

Este o metodă de atac care îl face pe “atacat” să ruleze cod injectat în loc de codul lui prin suprascriere a unor code pointeri.

42. Ce înseamnă memory leak / memory disclosure? De ce este acest lucru relevant pentru un atacator?

Înseamnă că cineva care nu ar trebui să aibă acces la niște date/adrese, are. Este util pentru atacator pentru a afla date personale sau alte porțiuni din memorie de interes. De asemenea poate deturna fluxul de execuție.

43. De ce în general, preferăm o împărțire a spațiului virtual de adrese între kernel space și user space? Și nu un spațiu dedicat pentru kernel space?

Se preferă o separare între kernel space și user space pentru a proteja sistemul de operare și hardware-ul de programe rău intenționate sau cu erori. Dacă un astfel de program rulează în user-space cu erori, acesta este blocat de sistemul de operare.

44. Ce secvență de cod C va duce la o excepție de acces la memorie (de tip Segmentation fault)? De ce?

```
int *a = malloc(7);
free(a);
*a = 2;
```

sau

```
char *p = NULL;
p = 2;
```

Se primește eroare de tip segmentation fault deoarece se încearcă accesarea unei zone de memorie invalidă.

45. **Cu ce diferă o funcție de o variabilă într-un executabil și/sau în cadrul spațiului de adrese al unui proces?**

    Funcția are permisiuni read-execute iar variabila are permisiuni read-write. De asemenea se poate altera fluxul de execuție al programului prin modificarea adresei de retur a funcției.

46. **Două procese partajează o zonă de memorie. Cum se manifestă acest lucru în tabelele de pagini ale celor două procese?**

    Când există o intrare comună la acea zonă de memorie, această schimbare va fii vizibilă pentru ambele procese. Ele au aceeași memorie fizică.

47. **Putem avea mai multă memorie fizică decât dimensiunea maximă a spațiului virtual de adrese al unui proces? Dar invers?**

    Da. Se poate și invers prin intermediul swapping ului.

48. **Ce zone de memorie se aloca static? Dar dinamic?**

    Zone statice: .text, .data, .rodata, .bss;
    Zone dinamice: heap, stivă.

49. **Ce se întâmplă cu o variabilă modificată într-un proces copil din perspectiva procesului părinte?**

    Părintele poate observa că variabila a fost modificată dar acest lucru nu îl afectează deoarece ei au pagini fizice separate. Dacă zonă de memorie este partajată, atunci schimbarea îl va afecta pe părinte.

50. **Ce reprezintă un loader? Ce rol are acesta?**

    Un loader este o componentă software care se ocupă cu încărcarea și configurarea programelor în memorie. Roluri:
    - alocarea memoriei necesare
    - încărcarea fișierelor executabile în memorie
    - realizarea legăturilor simbol-adrese

51. **La ce folosim apelul mprotect()? La ce mecanism de securitate putem face bypass folosind acest apel?**

    mprotect() modifică permisiunile de acces. mprotect() poate face bypass la bit-ul **NX**(no-execute este o tehnologie folosită de procesoare pentru a separa zone de memorie ce conțin date sau instrucțiuni folosite de procesor) și poate scrie și executa un posibil cod malițios.

52. **În ce zonă de memorie se află o variabilă globală, inițializată cu valoarea 0?**

bss

Calcul

1. **De ce este nevoie de procese?**

   Un proces este o instanță a unui program în execuție și o abstractizare făcută de sistemul de operare pentru o serie de acțiuni de multitasking și securitate.

   Ex: browserul care are nevoie de mai multe procese pentru performanță (încărcarea paginilor mai repede), deși folosesc mai multă memorie ram / cpu (idk)

2. **De ce este nevoie de thread-uri?**

   Un proces este împărțit în mai multe thread-uri, ele fiind cele care rulează pe procesor. Thread-urile sunt mai rapide deoarece ele au același VAS.

3. **Ce este un proces?**

   Un proces este un program în execuție.

4. **Ce este un thread?**

   Un proces este format din unul sau mai multe thread-uri.

   Conceptul de **thread** (fir de execuție) definește cea mai mică unitate de procesare ce poate fi programată spre execuție de către sistemul de operare. Este folosit în programare pentru a eficientiza execuția programelor, executând porțiuni distincte de cod în paralel în interiorul aceluiași proces.

5. **Cu ce diferă un thread de un proces?**

Proces	Thread
Independent	Parte dintr-un proces
Au VAS izolat	Împart VAS-ul cu alte thread-uri din proces
Se crează mai lent	Se crează mai repede
Durează mai mult sa își schimbe starea	Se schimbă mai repede starea
Dacă se termină procesul se termină toate thread-urile	Dacă se termină un thread, alte thread-uri continuă

6. **Cum este afectat spațiul virtual de adrese al unui proces în momentul creării unui thread?**

   Îi crește VAS-ul procesului cu dimensiunea stivei private a thread-ului. În unele cazuri, sistemul de operare poate implementa thread-uri folosind o stivă partajată în care VAS-ul procesului nu ar crește dacă se crează un thread nou.

7. **Ce zone de memorie au comune thread-urile unui proces și ce zone au specifice?**

   Fiecare thread are specific stack-ul și **TLS**(Thread Local Space). Ele au în comun tot în afară de stack si TLS.

   (Thread local space - the method by which each thread in a given multithreaded process can allocate locations in which to store thread-specific data.)(e un fel de variabila globală a threadului respectiv)

8. **Ce conține PCB (Process Control Block)?**

   - Starea procesului
   - PID-ul
   - Program counter: următoarea instrucțiune de executat
   - Regiștri de CPU
   - Planificarea informației pe CPU
   - Accounting information: cât la sută din procesor este folosit
   - I/O status information: Esta información incluye la lista de dispositivos de E/S asignado al proceso, una lista de archivos abiertos, etc.

   El dani mocano esta bueno en la puscaria. El Cobra Tate esta el major comando el dani mocano es le capybara mangiaro peach.
   Calin está sudoroso.

9. **Care sunt stările în care se poate găsi un thread?**

   ready, running, waiting, terminated.

10. **Ce efect are apelul fork()?**

    Tradițional, fork() crea un spațiu fizic și virtual nou pentru fiecare proces copil, adică se făcea un duplicat al spațiului fizic și se popula în tabela de pagini a procesului copil nou creat. De când a apărut conceptul **copy-on-write**, imediat după fork fiecare spațiu virtual referă la același spațiu fizic, deci nu se mai alocă spațiu fizic suplimentar. Atunci când unul dintre procese scrie, frame-ul este duplicat.

11. **Ce resurse partajează/nu partajează procesul părinte și procesul copil în cazul apelului fork()?**

Partajează toate resursele, însă procesul copil are propria stivă astfel încât procesul făcut de acesta să nu afecteze procesul părinte.

12. Cum modifică fork() si exec() spațiul virtual de adrese?

fork() - Procesul copil are un VAS identic (dar nou) cu cel al părintelui. (deep copy)
exec() - înlocuiește codul procesului și VAS-ul acestuia cu unul nou.

13. Ce efect are apelul exec()?

exec() înlocuiește memoria procesului curent cu un program nou. Loader-ul încarcă fișierul binar în memorie(distrugând imaginea memoriei procesului vechi) și îi începe execuția.

14. Cate threaduri se pot găsi în starea RUNNING, READY și WAITING?

În running se pot găsi atâtea thread-uri câte core-uri avem.
În ready se pot găsi oricâte thread-uri.
În waiting se pot găsi oricâte thread-uri(aici se găsesc cele mai multe).

15. Ce este o schimbare de context? Ce se întâmplă la o schimbare de context?

O schimbare de context este procesul prin care sistemul de operare își schimbă atenția de la un proces la altul. La o schimbare de context se salvează starea curentă a procesului care rulează și se încarcă starea altui proces pentru a putea fi executat.

16. Ce cauzează schimbări de context?

Running -> ready: când îi expiră unui proces cuanta de timp;
Ready -> running: când se eliberează un loc pe procesor;
Running -> waiting: când se realizează o operație I/O;
Waiting -> ready: când operația I/O blocantă s-a definitivat.

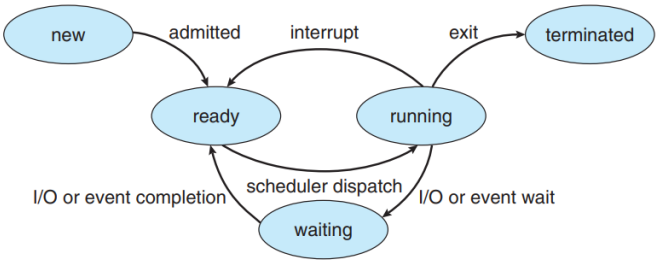


Figure 3.2 Diagram of process state.

17. Ce este o schimbare de context voluntară și o schimbare de context nevoluntară?

O schimbare de context voluntară este întâlnită la operație intensivă I/O(au o cuantă mai mare de timp decât alea CPU intensive, când intră în running, ele trec direct în waiting pentru a face loc altor procese cât timp așteaptă răspunsul).
O schimbare de context nevoluntară are loc la procesele CPU intensive, acestea rulând până când se termină cuanta de timp și sunt trimise în ready.

18. **Ce sunt threadurile cu implementare user-level și thread-urile cu implementare kernel-level?**

    Thread-urile cu implementare user-level sunt implementate la nivelul aplicației și sunt gestionate de bibliotecile de sistem ale aplicației. Acestea sunt gestionate fără intervenția sistemului de operare și sunt mai lente.
    Thread-urile cu implementare kernel-level sunt create și gestionate de sistemul de operare și au o performanță mai bună deoarece comunicarea cu sistemul de operare se face direct fără întreruperi.

19. **În ce situație este utilă zona TLS (thread local storage)?**
    Florin e calin sau dutu

    TLS e utilă dacă un thread are nevoie de variabile globale proprii.

    Calin esta sudoroso
20. **De ce este necesară sincronizarea proceselor/thread-urilor?**

    Este necesară sincronizarea proceselor pentru a evita suprascrierea datelor între procese.
    Pentru a evita deadlock-ul și race condition care apar când două sau mai multe operații sunt efectuate simultan.

21. **Ce înseamnă race condition?**

    Race condition este o situație în care mai multe procese accesează și manipulează aceleași date în același timp iar rezultatul execuției depinde de ordinea particulară în care are loc accesul.

22. **Ce înseamnă deadlock(cockblock)?**

    Deadlock este o situație în care două sau mai multe procese sunt blocate în așteptarea unui eveniment care nu se va produce deoarece alt proces depinde de același eveniment. Uneori un thread în waiting nu se va mai putea schimba niciodată deoarece resursele solicitate sunt deținute de thread-uri în waiting. (ex din carte: Intr-o țară a existat legea că dacă 2 trenuri se întâlnesc, nu pot pleca până nu pleacă celălalt (sau regula 1 la rocket league)).

23. **Care sunt dezavantajele sincronizării?**

Performanță scăzută (thread-urile trebuie să aștepte pentru a accesa resursele partajate), **complexitate crescută și costuri crescute**. De asemenea, sincronizarea poate duce la blocare dacă nu este implementată cum trebuie.

24. **Ce înseamnă TOCTOU (time of check to time of use)?**

    TOCTOU este un tip de race condition care implică verificarea unei valori variabile partajate/nepartajate după ce aceasta a fost modificată.

25. **Când se blochează un producător în problema producator-consumator? Dar un consumator?**

    Un producător se blochează când buffer-ul este plin iar un consumator se blochează când buffer-ul este gol.

26. **De ce este necesară prezența unei instrucțiuni de tipul atomic_compare_and_swap în fiecare ISA?**

    Instrucțiunea atomic_compare_and_swap este prezentă pe majoritatea ISA(instruction set architectures) pentru a permite implementarea unor mecanisme de sincronizare adecvate și pentru a asigura integritatea datelor în sistemele care permit accesul la aceleași date.

27. **Cu ce diferă un spinlock de un mutex? Când folosim spinlock-uri? Când folosim mutex-uri?**

    Spinlock este un lock intr-o bucla(folosește busy waiting). În cazul **spinlock**-ului, thread-ul se află în running în timp ce el așteaptă resursa necesară acesta ținând procesorul ocupat.
    Mutex este un sleeping lock. În cazul **mutex**-ului, thread-ul este băgat în waiting până când acesta primește resursa necesară. Acesta lasă procesorul să facă și altceva în timp ce el așteaptă resursa.
    Spinlock-ul este folosit pentru regiuni critice de mici dimensiuni în care nu se fac operații blocante.
    Mutex-ul este folosit pentru regiuni critice mai mari sau când se fac operații blocante.
    Dacă am folosi **mutex** pentru o regiune critică mică atunci overhead-ul cauzat de **mutex**(din cauza contex switch-ului) ar fi semnificativ mai mare față de timpul scurt petrecut în regiunea critică.

    Nu exista mutex intre **procese,** doar intre threaduri. Pentru procese se folosește **semafor** sau **file locking.**

    Obs: **Mutex** vs **Semafor**. Mutex: un singur thread poate folosi resursele simultan pe când la **Semafor**, exista un număr de thread-uri finit care pot folosi resursele simultan. Când un thread termină, trimite un semnal(notify) care incrementează nr de thread-uri ce pot folosi resursa.(ex cu baia)

    Deci semaforul este un mutex pe trenbolon.

Semafoarele sunt obiecte de sincronizare ce reprezintă o generalizare a mutex-urilor prin aceea că **salvează numărul de operații de eliberare** (incrementare) efectuate asupra lor. Practic, un semafor reprezintă un întreg care se incrementează/decrementează atomic. Valoarea unui semafor nu poate scădea sub 0. Dacă semaforul are valoarea 0, operația de decrementare se va bloca până când valoarea semaforului devine strict pozitivă. Mutex-urile pot fi privite, așadar, ca niște semafoare binare.

28. Ce efect are folosirea operatorului & din shell în crearea unui proces?

Îl trimite în background.

29. Ce este un proces zombie? Cum apare un proces zombie? Care este problema proceselor zombie?

Un proces zombie este un proces care și-a terminat execuția dar nu a fost așteptat de părinte. Procesele zombie nu-și eliberează memoria, ele așteptând să le fie luate datele, ocupând memoria foarte mult timp.

30. Ce este un proces orfan? De ce un proces este orfan foarte puțin timp?

Un proces este orfan în momentul în care părintele și-a terminat execuția. Procesele orfane sunt adoptate de **init**. **Init** va primi datele pe care copilul voia să le returneze.

31. Poate fi un proces zombie orfan? Ce se întâmplă cu un proces zombie orfan?

Dacă părintele moare și nu și-a așteptat copilul zombie acesta devine zombie orfan. El este adoptat de **init (ăsta e RD)** și este curățat de sistem. **Init** va primi datele pe care copilul voia să le returneze.

32. Ce se întâmplă dacă un thread realizează un acces invalid la o zonă de memorie?

Segmentation fault și este terminat.

DAR, în cazul în care se face demand paging la accesul invalid, se generează un page fault, iar dacă acesta are permisiunile necesare se creează pagina(înseamnă că acesta avea pagina rezervată) și se mapează, altfel procesul este terminat prin segmentation fault.

33. Poate un thread să acceseze stiva altui thread? Cum?

Da, se poate prin intermediul mecanismelor de sincronizare și comunicare oferite de sistemul de operare. Se poate folosi un semafor pentru accesarea unei zone comune care poate include stiva altui thread.

34. **De ce schimbarea de context între două thread-uri ale aceluiași proces este mai rapidă decât schimbarea de context între două thread-uri din procese diferite?**

    Deoarece au VAS-ul comun și nu se face TLB flush. (nu trebe să tragă apa la gunoi)

35. **Ce limitează numărul maxim de threaduri care pot fi create în cadrul unui proces?**

    Sistemul de operare, hardware și memoria disponibilă (probabil).

36. **Ce limitează numărul maxim de procese care pot fi create în cadrul unui sistem de calcul?**

    Sistemul de operare, hardware și RAM-ul disponibil. De regulă, sistemul de operare permite crearea unui număr mare de procese, dar acesta este influențat de performanță și resursele disponibile.

37. **Ce se întâmplă când toate procesele sistemului sunt blocate?**

    Dacă toate procesele sunt blocate, sistemul poate ajunge într-o stare inactivă (idle), nu mai poate răspunde la nicio solicitare și nu mai poate executa nicio activitate.

38. **Ce înseamnă waiting time (timp de așteptare) în planificarea proceselor?**
    Ce tristă e amalia
    Timpul pe care un proces l-a petrecut în ready.

39. **Putem avea un sistem multi-core cu un singur proces aflat în starea RUNNING și mai multe procese în READY?**

    Da, dacă pe procesor sunt thread-urile aceluiași proces. Windows este optimizat astfel încât core-urile să nu stea libere cât timp exista procese în waiting. În linux se poate din cauza unui bug(din ce zicea Radovici). Dar teoretic nu se poate.

40. **Ce înseamnă starea READY/RUNNING/TERMINATED/WAITING(BLOCKED)?**

    Ready - așteaptă să ruleze
    Running - rulează
    Terminated - a terminat execuția sau a fost întrerupt de sistemul de operare
    Waiting - așteaptă să primească resursele necesare rulării (pt I/O)

41. **Ce este un proces I/O intensive?**

    Un proces care primește o cuantă mai mare de timp pentru că vor trece din running în waiting și vor lăsa loc altui proces pe CPU.

42. **Ce este un proces CPU intensive?**

Un proces care primește o cuantă mai mică de timp. Este posibil să ruleze pe procesor până la expirarea cuantei. Dacă ar avea o cuantă mare de timp ar dura mult până ar lăsa loc altui proces.

43. Cum tratează planificatorul procesele I/O intensive și procesele CPU intensive?

Vezi mai sus

44. Două thread-uri ale unui proces execută aceeași funcție. Care sunt diferențele între cele două thread-uri?

Starea, code pointer-ul, stiva și TLS-ul.

45. Cu ce diferă procesul copil (creat prin fork) de procesul părinte?

Are un pid nou, are stivă proprie, are VAS separat, starea, prioritatea procesului, fișierele deschise (la început sunt aceleași dar el poate scrie/închide/deschide independent).

46. Cu ce diferă un proces zombie de un proces orfan?

Zombie	Orfan
A terminat execuția	Nu a terminat execuția
Așteaptă să îi dea părintele wait	Așteaptă să fie adoptat de init
Are părinte	Nu are părinte (lmao) - a plecat dupa lapte

47. Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cu productivitate mai mare?

Prioritatea proceselor, algoritmul de planificare, intervalul de timp pentru planificare, nr de procese în ready(un nr mare îmbunătățește productivitatea).
Procesele CPU intensive primesc o cuantă de timp mai mare.

48. Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cât mai interactiv (responsive)?

Prioritatea proceselor, algoritmul de planificare, intervalul de timp pentru planificare, nr de procese în ready, prioritatea celor sensibile la întârzieri(I/O).
Procesele I/O intensive primesc prioritate mare.

49. Am avea nevoie de folosirea unui apel de sistem pentru crearea unui thread în cazul unei implementări de tip user-level threads? Dar în cazul deschiderii unui fișier în același scenariu?

Pentru crearea unui thread la user-level nu este nevoie de apel de sistem deoarece sunt implementate prin intermediul bibliotecilor de sistem.
Pentru deschiderea unui fișier poate fi utilizat apelul open() care este un apel de sistem.

50. Ce se întâmplă dacă folosim apeluri de sistem blocante în interiorul unui spinlock?

Se poate produce deadlock deoarece se așteaptă după ceva în waiting. De asemenea și alte thread-uri care au nevoie de resursele protejate de spinlock se pot bloca.

(gen se referă că celelalte thread-uri așteaptă după ăsta care rulează, dar ăsta e băgat în waiting din cauza apelului blocant)

51. De ce este necesară folosirea prefixului LOCK pentru realizarea operațiilor atomice?

Prefixul LOCK este utilizat pentru a indica că o instrucțiune atomică trebuie să fie executată cu ajutorul unui mecanism de sincronizare. Acesta se face pentru a preveni race condition. De exemplu, în cazul unui sistem multicore, o instrucțiune de tip lock poate fi utilizată pentru a preveni accesul concurent la zona de memorie de către mai multe procese.

52. Care este avantajul folosirii mutexurilor în defavoarea spinlockurilor și invers?

Mutex se folosește pentru o regiune critică mai mare în timp ce spinlock se folosește pentru o regiune critică mai mică fără apeluri blocante.
Avantajul mutex-ului este folosirea procesorului în timpul așteptării după resursele necesare.
Avantajul spinlock-ului este viteza pe porțiuni mici.

53. Care este echivalentul apelului wait() pentru thread-uri?

Apelul join()(sau poate pthread_join()). Apelul join permite unui thread să aștepte finalizarea unui alt thread pentru a putea accesa rezultatele și a continua execuția.

54. De ce este nevoie de apelul pthread_wait() / pthread_join()?

Apelul pthread_join() permite unui thread să aștepte finalizarea unui alt thread pentru a putea accesa rezultatele și a continua execuția. Este necesar pentru a gestiona ordinea execuției și pentru a preveni race condition.

55. Ce tranziție între stările unui thread nu este posibilă?

Waiting->running
Ready->waiting
Waiting/ready->terminated

I/O

1. **Ce conține un FCB (File Control Block)?**
   Conține:
   -identificator (fd)
   -controlul accesului (permisiunea?)
   -timp de acces
   -tip de fișiere
   -dimensiunea datelor
   -pointer către date
   Obs: Nu conține numele, acesta fiind ținut în dentry(Directory entry).

2. **Ce reprezintă un descriptor de fișier?**

   Un număr care referă o intrare în tabela de descriptori de fișiere. Este folosit în operații de lucru cu fișiere pentru a identifica un fișier deschis.

3. **Ce reprezintă tabela de descriptori de fișiere?**

   O tabelă în care se țin file descriptorii. Ca structură de date este un vector de pointeri.

4. **Câte tabele de descriptori de fișiere se găsesc într-un sistem de operare?**

   Unu per proces.

5. **Ce efect are apelul dup()?**

   Apelul dup() duplică un file descriptor pe prima poziție validă din FDT (file descriptor table). Noul descriptor va pointa către acea structură de file descriptori ca și descriptorul pe care l-a duplicat. Redirectează și alege primul file descriptor liber din fdt.
   dup2() este pentru redirectarea intrării/ieșirii/erorii.

6. **Ce efect are apelul close()?**

   Închide file descriptor-ul și șterge referința programului către acesta. În cazul în care file descriptorul este ultimul care pointează la structura de file descriptori deschisă de open, atunci toate resursele asociate cu file descriptorul curent se eliberează (sau ceva de genu).

   Dacă s-a folosit unlink() pe ultimul file descriptor, fișierul se va șterge în urma apelării close.

7. **Ce apeluri modifică pointer-ul/cursorul de fișier (file pointer)?**

   fseek(), lseek(), read(), write()

8. **Ce apeluri modifică dimensiunea fișierului?**

   ftruncate(), write() sau open() cu argumentul O_TRUNC(reduce dimensiunea fișierului la 0).

9. **Ce efect are apelul/comanda truncate?**

   Trunchiază fișierul cu dimensiunea specificată.
   (If the file previously was larger than this size, the extra data is lost. If the file previously was shorter, it is extended, and the extended part reads as null bytes ('\0').)

10. **Care sunt tipurile de fișiere pe un sistem de fișiere uzual Unix?**

    Regular, director, symbolic link, FIFO (pipe), caracter special, socket și block special.

11. **Ce este un sistem de fișiere virtual?**

    Soft care permite accesul la fișierele de pe disk in mediul virtual.

    VFS este o modalitate prin care aplicațiilor din user space le este pusă la dispoziție o interfata a sistemului de fișiere de pe kernel

12. **Ce este un dispozitiv virtual?**

    Un dispozitiv virtual pointează către un device file care nu are hardware asociat. Deși acest dispozitiv nu are hardware, sistemul de operare crede că are. Pe linux putem crea un dispozitiv virtual folosind comanda mknod.

    De exemplu, un bug sau un virus pot fi detectate presupunând că un dispozitiv extern îl monitorizează.

13. **Ce tipuri de dispozitive cunoașteți? Clasificați-le din orice punct de vedere cunoașteți**

    Dispozitivele caracter se ocupă cu gestionarea datelor one character at a time. Exemple de dispozitive sunt tastatura, mouse-ul, placă de sunet etc.

    Dispozitivele blocuri sunt responsabile cu gestionarea datelor one block at a time. Dimensiunea unui bloc este în general multiplu de 512 exprimată în bytes. Exemple sunt hard disk drive-urile(HDD / SSD), CD-ROM drive-urile, RAM disk-urile

14. **Cu ce diferă un dispozitiv de tip bloc de un dispozitiv de tip caracter? Dați câte un exemplu de fiecare.**

Dispozitivul de tip caracter nu are nevoie de buffering, driverul asociat este mai simplu, necesită mai puțin efort fiindcă are de gestionat o singură porțiune. Apelurile read() și write() sunt blocante.

Dispozitivul de tip bloc are nevoie de buffer, driver-ul asociat necesită mai multă atenție având responsabilitatea de a naviga printre bucăți mari de blocuri în cadrul mediului de stocare. Apelurile bread() și bwrite() pot fi asincrone.

15. De ce nu are sens operația de seek pe un dispozitiv de tip caracter?

Nu se poate modifica cursorul deoarece acest tip de dispozitiv are o singură poziție(cea curentă).

16. Ce adresă IP locală și ce port local are un socket întors de apelul accept()?

Funcția returnează file descriptor-ul nou creat care conține adresa/portul asociat peer-ului(aplicația căreia îi permitem să se conecteze).

accept() acceptă conectarea la un socket și întoarce fd pentru socketul respectiv cu adresa IP a rețelei locale și portul aplicației care dorește sa se conecteze.

*accept() returnează un socket pe care serverul îl folosește pentru comunicarea cu clientul, IP address 127.0.0.1 on port 6013 (nu știu așa scrie în carte).

17. Ce valoare poate întoarce un apel read() sau un apel write()?

Returnează numărul de bytes citiți/scriși sau 0 în cazul în care **read** nu a citit nimic sau -1 în cazul în care **write** a dat eroare.

18. Ce operații se pot face pe fișiere?

-open()
-close()
-read()
-write()
-lseek()

19. Ce operații asupra fișierelor modifică/nu modifică valoarea cursorului unui fișier?

Care modifică:
-lseek()
-read()
-write()
Care nu modifică:
-open()
-close()
-creat()

20. Ce operații asupra fișierelor modifică/nu modifică dimensiunea fișierului?

Modifică: truncate și write(dacă scrii peste dimensiunea curentă), open() cu parametru (O_TRUNC - reduce dimensiunea fișierului la 0).
Nu modifică: lseek(), read(), close(), open() fara parametru

21. Unde este reținută valoarea cursorului de fișiere (file pointer) și unde este reținută dimensiunea fișierului?

Dimensiunea fișierului este reținută în inode(sau fcb) iar file pointer-ul este reținut de către sistemul de operare la nivelul kernel-ului.

22. De ce avem două buffere asociate fiecărui socket, ce rol are fiecare?

Unu trimite, unu primește.

23. Ce este o operație asincronă?

O operație asincronă este o operație care poate fi executată în paralel cu alte operații fără a bloca execuția programului. Aceasta permite programului să efectueze mai multe operații în același timp, crescând astfel eficiența și performanța.

Ex: web page load

24. Ce este o operație neblocantă?

Este o operație care trece în wait până când primește resursele necesare rulării, astfel neblocând celelalte operații.

25. Ce întoarce o operație asincronă?

Eroare sau lucrul dorit.

26. Ce întoarce o operație blocantă?

O operație blocantă întoarce rezultatul acțiunii. Tocmai de aceea este blocantă, ea așteaptă până la încheierea taskului.

27. Cu ce diferă un socket de rețea de un socket UNIX?

Un socket de rețea este folosit pentru comunicarea între sisteme diferite și socketul Unix este folosit pentru comunicarea locală. Deci socket-ul de rețea e pe trenbolon.

28. Care este diferența între un pipe anonim și un pipe cu nume (named pipe)?

Pipe-ul anonim poate comunica doar într-o relație părinte-copil, adică procese înrudite în timp ce pipe-ul cu nume(FIFO) permite comunicarea între procese neînrudite.

29. Ce este buffer cache-ul? Care este rolul său?

La lucrul cu fișiere, apelurile read și write copiază întâi toate datele într-un buffer din kernel-space numit buffer cache. Ele apoi sunt căutate întâi în acest buffer înainte de a le căuta pe disk. Acest lucru mărește performanța sistemului deoarece căutarea pe disk este mult mai lentă.

Buffer caches are located in the RAM (Random Access Memory) of a computer. They are used to store data temporarily so that the CPU can access it quickly.

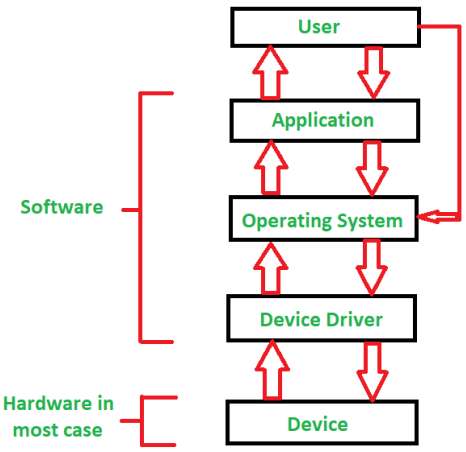
30. De ce operația write pe fișiere este foarte rar blocantă?

Operația de write ar fi blocantă dacă bufferul din kernel ar fi plin. Acest caz este extrem de rar deoarece dacă buffer-ul se umple atunci el este fie eliberat prin scrierea pe disk a informațiilor pe care le reține, fie mărit.

31. În ce situație operația read() pe fișier se blochează?

Operația de read() este blocantă în cazul în care bufferul din kernel-space este gol. Acțiunea va fi blocantă până când un byte ajunge pe buffer.

32. Care este rolul unui device driver?



Permite unui device hardware să comunice cu sistemul de operare. Este o componentă software.

Ex: draivăr de amprimantă

33. Ce rol are controller-ul hardware?

Controlează componente hardware. Are rolul de administrator. Este la rândul său o componentă hardware care permite transmiterea, flow-ului de date, între mai multe entități fizice.

E o componenta hardware sau un program software care ajuta dispozitive hardware să comunice.

34. Ce înseamnă zero-copy? Ce mecanism/apel folosește zero-copy?

Presupune transferul de informații dintre două sau mai multe buffere din kernel-space fără a se trece prin user-space.
Apelurile folosite pentru zero-copy:
-send file
-splice (zis de amalia, dec-i nu e bn 🙁)
-transmite file(windows)

35. Ce garanții ni se oferă în momentul în care apelul send() se întoarce în user space?

Send este folosit pentru a scrie mesaje între socket-uri(returnează nr de bytes scrișsau -1 în cazul unei erori). Este echivalentul write-ului doar că acesta are și flag. Dacă se întoarce în user-space, avem garanția că datele au fost copiate într-un buffer, dar nu avem garanția că datele au fost primite de destinatar.

36. Cu ce diferă afișarea folosind printf() față de folosirea write()?

printf() folosește un buffer în user-space care este golit din când în când(în general la new line sau când se umple), iar write() folosește un buffer în kernel-space.

37. De ce subsistemul de networking nu folosește buffer cache-ul?

Buffer cache-ul este folosit pentru a eficientiza scrierea/citirea datelor de pe disk. Subsistemul de networking nu folosește acest buffer cache deoarece acesta nu salvează date pe disk.

38. Ce rol are apelul / comanda sync?

Sync scrie datele din buffer cache pe disk.

39. Care este rolul apelului ioctl / DeviceIoControl?

ioctl() este un apel de sistem ce se ocupă cu comunicarea dintre aplicații și draivăre fără a “inventa” un nou apel de sistem. Știe ce sa faca în funcție de parametri pe care ii primeste.

40. De ce în general doar utilizatorul root are permisiuni de scriere (uneori doar root are permisiuni de citire) pe intrările din /dev?

Deoarece în /dev se află divais draivărele iar dacă utilizatorul neprivilegiat ar avea acces la ele ar apărea probleme de securitate și s-ar putea crea vulnerabilități în comunicarea cu dispozitivele de I/O.

41. **De ce este apelul fwrite() mai rapid decât write atunci când facem multe scrieri?**

    fwrite() combină toate apelurile într-un buffer și apoi face un singur apel mare de sistem. write() cheamă individual apelul de sistem de fiecare dată.

42. **Ce se întâmplă dacă facem open de mai multe ori consecutiv pe același fișier?**

    Dacă se face pe aceeași variabilă, se va pierde memorie.
    Dacă se face pe două variabile diferite, se creează doi file descriptori diferiți și se deschid ca și cum ar fi două fișiere separate.

43. **Cum se modifica tabelul de file descriptori după open() si dup()? Este echivalent cu doua apeluri open()?**

    dup(old_fd)

    open() crează un nou file descriptor și un nou open file structure în FDT pe când dup() crează un fd nou pe prima poziție validă din FDT și duplică open file structure-ul fd-ului vechi pe care îl primește ca argument.

    dup() nu este echivalent cu două open() deoarece el nu crează open file structure, ci doar copiază referința celui vechi.

    open() face deep copy
    dup() face shallow copy Așa. punct/.

44. **Ce fișiere sunt deschise, în general, la crearea unui proces nou?**

    Stdin, stdout, stderr cu fd 0, 1, 2.

45. **De ce este utilă prezența unor dispozitive pur virtuale în ierarhia /dev (ex. */dev/vboxnetctl*, */dev/urandom*)?**

    Pentru a emula funcționalitatea unui dispozitiv fizic care nu este prezent. Pentru a avea funcționalități pe care un dispozitiv fizic nu le poate oferi.

46. **De ce, în general, apelul write() pe un fișier nu blochează threadul curent?**

    Deoarece acesta copiază memoria din buffer-ul din user-space în bufferul din kernel-space.

47. **În ce situație un apel read() pe un fișier blochează threadul curent?**

Când buffer-ul din kernel-space este gol.

48. **De ce spunem că operațiile read() / write() seamănă cu operația memcpy()?**

    Deoarece read și write copiază date din user-space în kernel-space(sau invers).

Interacțiunea între aplicații

1. **Ce forme de comunicare inter-proces cunoști?**

   Pipe-uri, sockets, semnale și memorie partajată

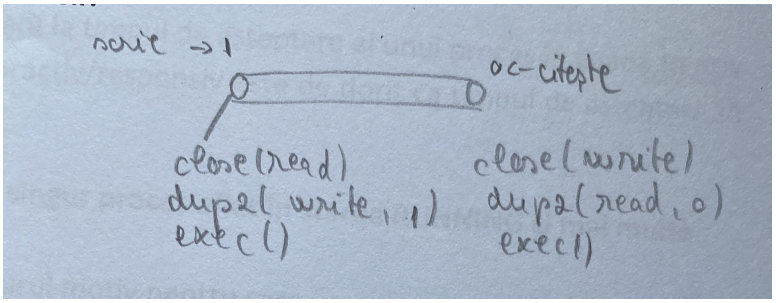
2. **Ce este un semnal? Când se trimite un semnal către un proces?**

   Este notificarea asincronă pe care un proces / sau so i-o trimite unui proces sau thread specific al său. De exemplu un copil când își termină execuția trimite un semnal părintelui seek child.

3. **Cine trimite un semnal unui proces?**

   Sistemul de operare sau alt proces punct.

4. **Cum este implementat operatorul | din shell?**



   Operatorul | din shell (sau operatorul de pipe) este utilizat pentru a conecta două comenzi, astfel încât rezultatul primei comenzi să devină intrarea pentru a doua comandă. Astfel, rezultatul primei comenzi este redirecționat ca intrare pentru comanda următoare.

   De exemplu, pentru a compara două fișiere, puteți utiliza comanda "diff":
   diff fisier1.txt fisier2.txt

   Sau, cu operatorul |, puteți face același lucru folosind:
   cat fisier1.txt | diff - fisier2.txt

5. **Care sunt avantajele și dezavantajele folosirii memoriei partajate pentru comunicarea inter-proces?**

   Avantaje:
   - cea mai rapidă metodă de comunicare între procese
   - mai puțină memorie folosită

   Dezavantaje:
   - pot exista probleme de sincronizare ceea ce ar duce la date incorecte

6. **Care sunt avantajele și dezavantajele pipe-urilor pentru comunicarea inter-proces?**

   Avantaje:
   - comunicare facilă și relativ rapidă între procese înrudite (pentru pipe-uri anonime) sau neînrudite (pipe-uri cu nume)
   - nu e nevoie de sincronizare (comunicarea se face unidirecțional)

   Dezavantaje:
   - au o singură direcție de comunicare
   - dimensiunea limitată a buffer-ului

7. **Care este limitarea folosirii pipe-urilor anonime?**

   Pot comunica doar cu procese înrudite.

8. **Care este diferența principală între FIFO și socket UNIX?**

   FIFO este unidirecțional pe când socket-ul UNIX este bidirecțional, acesta având un buffer de scriere și unul de citire. De asemenea, FIFO este implementat în user-space iar socket-ul UNIX este implementat în kernel-space.

9. **Ce primitive de interacțiune pot fi folosite între procese de pe sisteme diferite?**

   Doar socket-urile de rețea.

10. **Ce primitive de comunicare pot fi folosite doar între threaduri?**

    Doar shared memory. (VAS ???)

11. **Ce primitive de comunicare pot fi folosite doar între procese înrudite?**

    Pipe-uri anonime. Punct. !!!!. Punct ba ca zice doar baaa (shared memory ????)

12. **Cum pot două procese să folosească semafoare anonime?**

    Cu talent, sucarime si finete(dedicație venită tocmai din vieni)

fără nume - ce pot fi folosite pentru sincronizarea între firele de execuție ale aceluiași proces, sau între procese - cu condiția ca semaforul să fie într-o zonă de memorie partajată.

13. **Cum pot două procese neînrudite să folosească semafoare anonime?**

    Două procese neînrudite pot folosi semafoare anonime dacă semaforul este plasat într-o zonă share-uită de memorie.

    + Named semaphores have an owner user-id, group-id, and a protection mode.

14. **Ce rol are apelul mmap() în cazul partajării memoriei între două procese?**

    mmap() (prin folosirea flag-ului **MAP_ANONYMOUS / MAP_ANON same shit | MAP_SHARED**) creează zone de memorie anonime ce pot fi folosite de procese înrudite. Mai mult, mapează fișiere ce folosesc atât mecanismele de demand paging cât și de lazy loading pentru o folosire cât mai eficientă a memoriei.

15. **Ce mecanisme putem folosi pentru a asigura accesul exclusiv la o zonă de memorie partajată între două procese?**

    Semafoare

16. **De ce este nevoie de sincronizare la folosirea memoriei partajate, dar nu la folosirea unui socket pentru comunicarea între două procese?**

    În cazul memoriei partajate este nevoie de sincronizare pentru a evita problema concurenței dintre două thread-uri/procese.
    În cazul folosirii unui socket nu există această problemă pentru că acesta are două buffere.

17. **Ce înseamnă aplicație omogenă? Dați un exemplu.**

    O aplicație omogenă este o aplicație care folosește thread-uri/procese pentru a realiza același tip de acțiune concomitent. De obicei, astfel de aplicații dau dovadă de interacțiune minimală între thread-uri/procese, acestea doar dau join/wait la finalul execuției job-ului.

    Ex: paginile de browser din firefox(thread-uri) sau paginile de browser din chrome(procese)

18. **Ce înseamnă aplicație eterogenă? Dați un exemplu.**

    O aplicație eterogenă este o aplicație care folosește thread-uri/procese pentru a realiza tipuri diferite de acțiuni.De obicei, astfel de aplicații dau dovadă de interacțiune bogată între thread-uri/procese, sunt folosite mecanisme de IPC.

    Ex: gitlab, microsoft office

19. **Care este avantajul și dezavantajul distribuirii unei aplicații pe mai multe sisteme?**

    Avantaje:
    -se evită poluarea sistemului propriu
    -putere computațională mai mare

    Dezavantaje:
    -comunicare mai lentă între componente(se pot folosi doar sockets de rețea)

20. **Care este avantajul implementării unei aplicații în format multi-process față de format multi-threaded?**

    Avantajul implementării unei aplicații în format multi-process față de format multi-threaded este acela că fiecare proces este izolat de celelalte procese deoarece acesta are VAS-ul lui propriu. Totuși o aplicație multi-thread share-uiește VAS-ul ceea ce o face mai eficientă d.p.d.v al memoriei și al vitezei.

21. **Cum pot două procese partaja memorie anonimă? Adică folosind flagul MAP_ANONYMOUS la mmap(). Ăsta e flag de transpirați**

    Două procese pot partaja memorie anonimă dacă se folosește și flagul MAP_SHARED împreună cu MAP_ANONYMOUS. Acest lucru are următoarele efecte:
    -maparea și zeroizarea unei zone distincte de memorie
    -copiii moștenesc zonele mapate ale părinților
    -nu există copy-on-write când unul dintre procese încearcă să scrie
    -zonele mapate anonim sunt folosite ca mecanism de IPC, ceea ce numim shared memory

22. **Ce mecanism de interacțiune între aplicații au un buffer și ce mecanisme de interacțiune nu au un buffer?**

    Cu buffer: Pipe și UNIX sockets (probabil și alea de rețea)
    Fără buffer: Fișiere, mutex, shared memory, semafoare și semnale

    Inode windows = MFT (master file table)
    Mutex doar pt thread-uri

Pipe-urile (canalele de comunicație) sunt mecanisme primitive de comunicare între procese. Un pipe poate conține o cantitate limitată de date. Accesul la aceste date este de tip FIFO (datele se scriu la un capăt al pipe-ului pentru a fi citite de la celălalt capăt). Sistemul de operare garantează sincronizarea între operațiile de citire și scriere la cele două capete (Linux, Windows).

Socket-ul e ca pipe-ul, doar că are 2 buffere, fiind bidirecțional (pipe-ul e unidirecțional). Pipe-ul se folosește local în general din cauza rapidității și a costului redus, pe când socket-ul e folosit în rest. Socket-ul e mai bun decât pipe-ul, dar costă mai mult și are nevoie de sincronizare (din cauza celor 2 buffere).